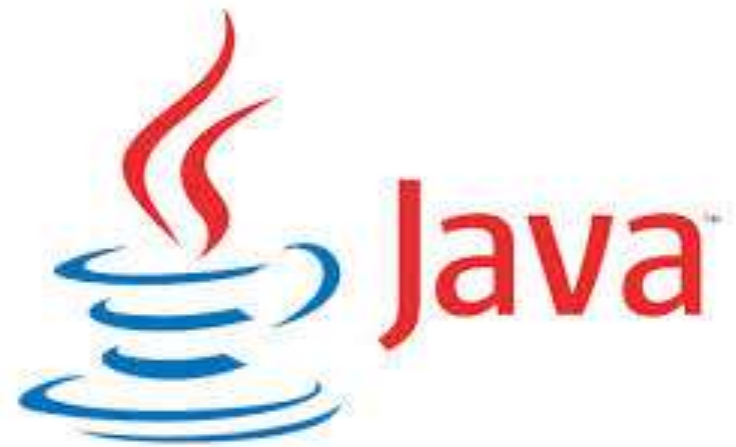




SOC2030

# Application Programming in Java



Week 6

Dr. Andrei Dragunov

# The String Class

## Key Point

*A **String** object is immutable: Its content cannot be changed once the string is created.*

## Constructing a String

You can create a string object from a string literal or from an array of characters.

- `String newString = new String(stringLiteral);`
- `char[] charArray = {'G', 'o', 'o', 'd', ' ', 'D', 'a', 'y'};`
- `String message = new String(charArray);`

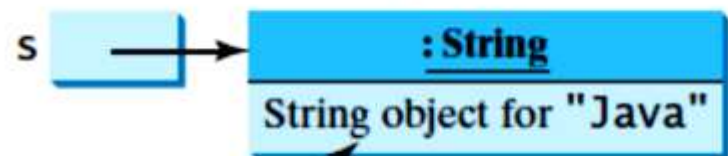
# Immutable Strings and Interned Strings

A **String** object is immutable; its contents cannot be changed. Does the following code change the contents of the string?

```
String s = "Java";  
s = "HTML";
```

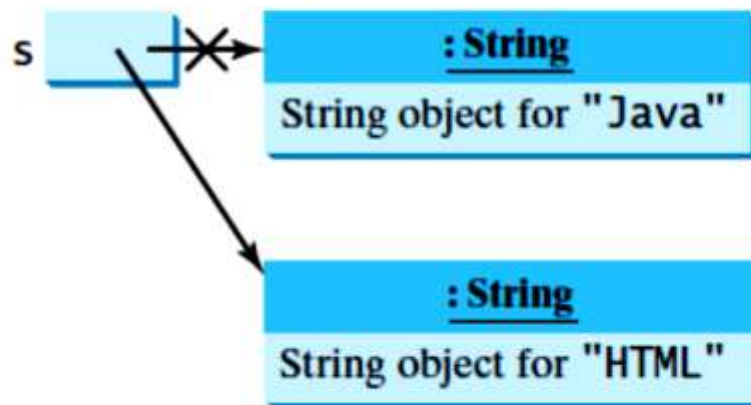
The answer is no. The first statement creates a **String** object with the content "**Java**" and assigns its reference to **s**. The second statement creates a new **String** object with the content "**HTML**" and assigns its reference to **s**. The first **String** object still exists after the assignment, but it can no longer be accessed, because variable **s** now points to the new object, as shown in next slide

After executing `String s = "Java";`



Contents cannot be changed

After executing `s = "HTML";`



This string object is now unreferenced

- Because strings are immutable and are ubiquitous in programming, the JVM uses a unique instance for string literals with the same character sequence in order to improve efficiency and save memory. Such an instance is called an *interned string*. For example, the following statements:

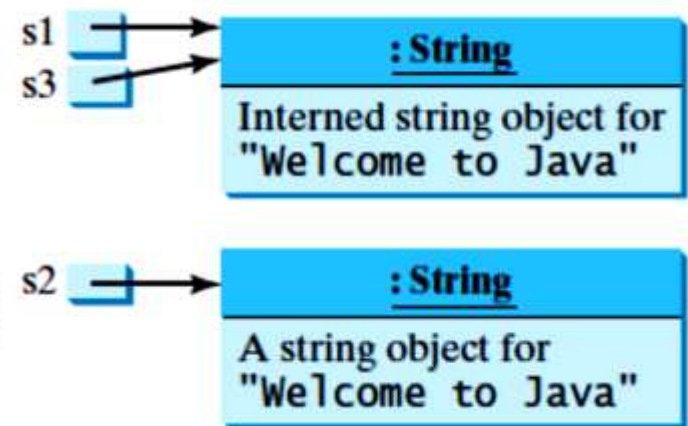
```
String s1 = "Welcome to Java";
```

```
String s2 = new String("Welcome to Java");
```

```
String s3 = "Welcome to Java";
```

```
System.out.println("s1 == s2 is " + (s1 == s2));
```

```
System.out.println("s1 == s3 is " + (s1 == s3));
```



# Matching, Replacing and Splitting by Patterns

- A *regular expression* (abbreviated *regex*) is a string that describes a pattern for matching a set of strings. You can match, replace, or split a string by specifying a pattern. This is an extremely useful and powerful feature.

**"Java is fun".matches("Java.\*")**

**"Java is cool".matches("Java.\*")**

**"Java is powerful".matches("Java.\*")**

# Converting Characters and Numeric Values to Strings

## **java.lang.String**

+valueOf(c: char): String

+valueOf(data: char[]): String

+valueOf(d: double): String

+valueOf(f: float): String

+valueOf(i: int): String

+valueOf(l: long): String

+valueOf(b: boolean): String

Returns a string consisting of the character **c**.

Returns a string consisting of the characters in the array.

Returns a string representing the **double** value.

Returns a string representing the **float** value.

Returns a string representing the **int** value.

Returns a string representing the **long** value.

Returns a string representing the **boolean** value.

# Design Patterns in Java

- **A design patterns are well-proved solution for solving the specific problem/task.**
- Design patterns represent the best practices used by experienced object-oriented software developers. Design patterns are solutions to general problems that software developers faced during software development. These solutions were obtained by trial and error by numerous software developers over quite a substantial period of time.

# Advantage of design pattern:

1. They are reusable in multiple projects.
2. They provide the solutions that help to define the system architecture.
3. They capture the software engineering experiences.
4. They provide transparency to the design of an application.
5. They are well-proved and testified solutions since they have been built upon the knowledge and experience of expert software developers.
6. Design patterns don't guarantee an absolute solution to a problem. They provide clarity to the system architecture and the possibility of building a better system.

# Categorization of design patterns:

Basically, design patterns are categorized into two parts:

- Core Java (or JSE) Design Patterns.
- JEE Design Patterns.

# Types of Design Patterns

| S.N. | Pattern & Description                                                                                                                                                                                                                                                                      |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>Creational Patterns</b><br>These design patterns provide a way to create objects while hiding the creation logic, rather than instantiating objects directly using new operator. This gives program more flexibility in deciding which objects need to be created for a given use case. |
| 2    | <b>Structural Patterns</b><br>These design patterns concern class and object composition. Concept of inheritance is used to compose interfaces and define ways to compose objects to obtain new functionalities.                                                                           |
| 3    | <b>Behavioral Patterns</b><br>These design patterns are specifically concerned with communication between objects.                                                                                                                                                                         |
| 4    | <b>J2EE Patterns</b><br>These design patterns are specifically concerned with the presentation tier. These patterns are identified by Sun Java Center.                                                                                                                                     |

# 1.Creational Design Pattern

- **Factory Pattern (Today)**
- Abstract Factory Pattern
- **Singleton Pattern (Today)**
- Prototype Pattern
- **Builder Pattern. (Friday)**

## 2. Structural Design Pattern

- **Adapter Pattern (Friday)**
- Bridge Pattern
- **Composite Pattern (Friday or self-study)**
- **Decorator Pattern (Today)**
- Facade Pattern
- Flyweight Pattern
- Proxy Pattern

# 3. Behavioral Design Pattern

- Chain Of Responsibility Pattern
- Command Pattern
- Interpreter Pattern
- Iterator Pattern
- Mediator Pattern
- Memento Pattern
- **Observer Pattern (Friday or self-study)**
- State Pattern
- Strategy Pattern
- Template Pattern
- **Visitor Pattern**

# Factory Pattern

- Factory pattern is one of the most used design patterns in Java. This type of design pattern comes under creational pattern as this pattern provides one of the best ways to create an object.
- In Factory pattern, we create object without exposing the creation logic to the client and refer to newly created object using a common interface.
- A Factory Pattern or Factory Method Pattern says that just define an interface or abstract class for creating an object but let the subclasses decide which class to instantiate. In other words, subclasses are responsible to create the instance of the class.
- The Factory Method Pattern is also known as Virtual Constructor.

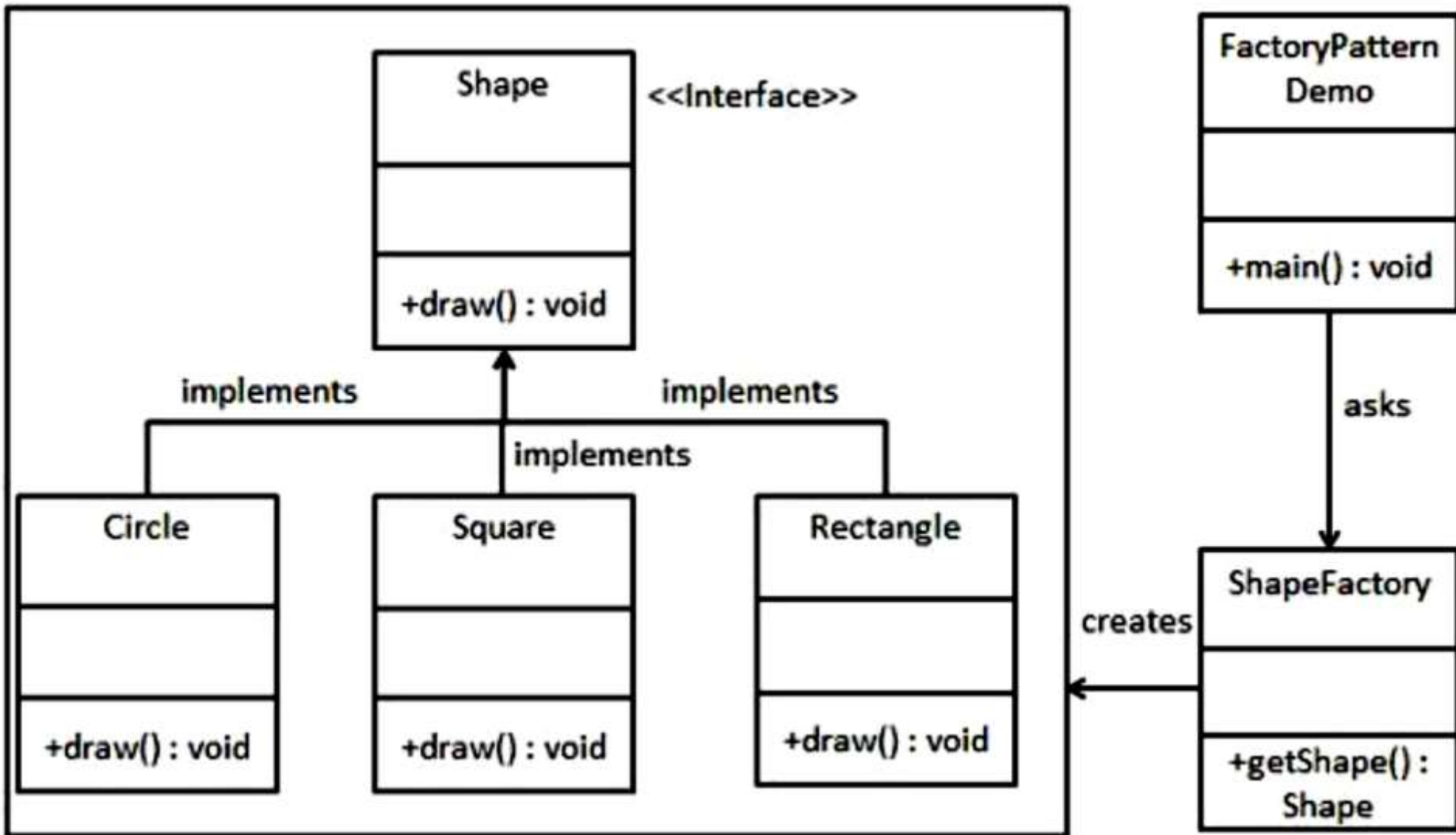
## Advantage of Factory Design Pattern

- Factory Method Pattern allows the sub-classes to choose the type of objects to create.
- It promotes the **loose-coupling** by eliminating the need to bind application-specific classes into the code. That means the code interacts solely with the resultant interface or abstract class, so that it will work with any classes that implement that interface or that extends that abstract class.

## Usage of Factory Design Pattern

- When a class doesn't know what sub-classes will be required to create
- When a class wants that its sub-classes specify the objects to be created.
- When the parent classes choose the creation of objects to its sub-classes.

# Implementation



# Singleton Pattern

- Singleton pattern is one of the simplest design patterns in Java. This type of design pattern comes under creational pattern as this pattern provides one of the best ways to create an object.
- This pattern involves a single class which is responsible to create an object while making sure that only single object gets created. This class provides a way to access its only object which can be accessed directly without need to instantiate the object of the class.

There are two forms of singleton design pattern

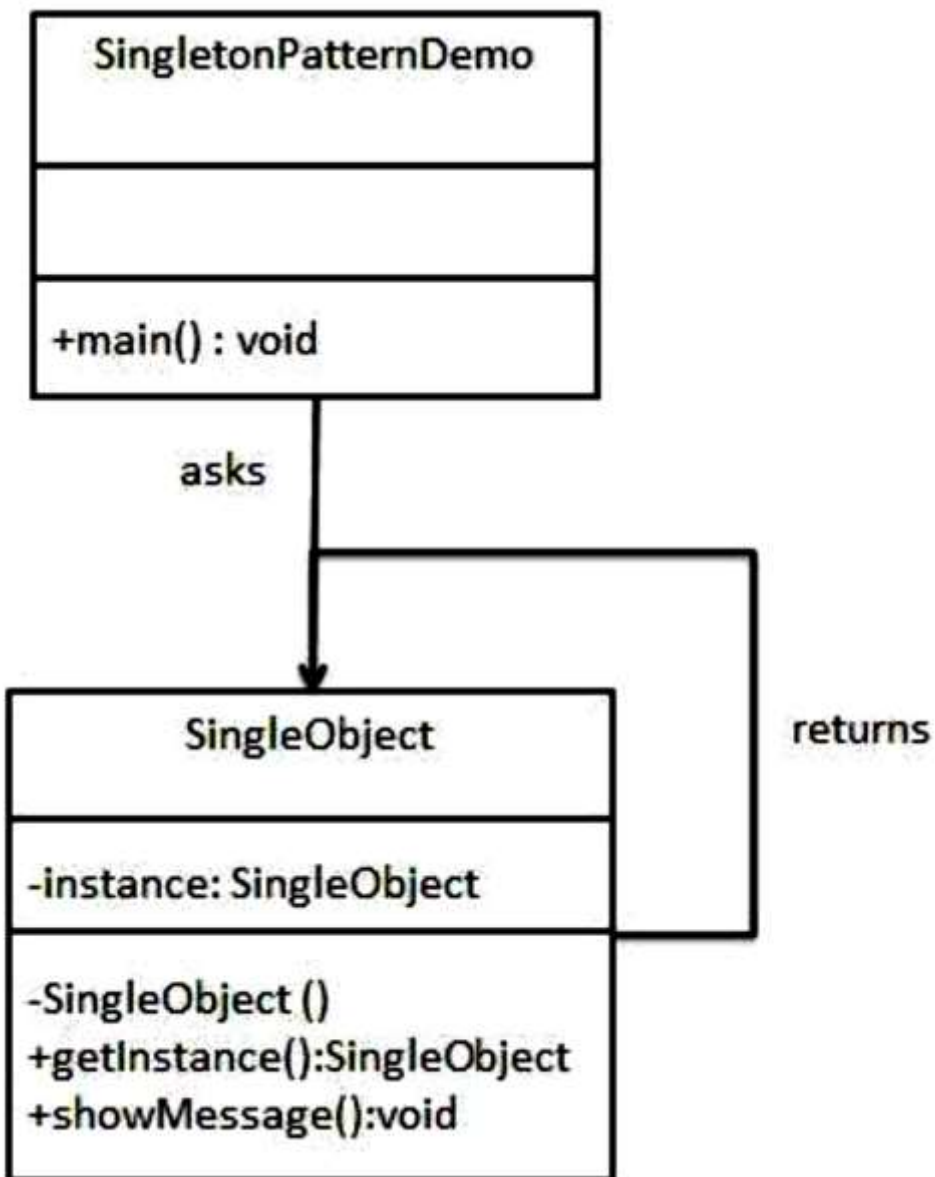
- **Early Instantiation:** creation of instance at load time.
- **Lazy Instantiation:** creation of instance when required.

## **Advantage of Singleton design pattern**

- Saves memory because object is not created at each request. Only single instance is reused again and again.

## **Usage of Singleton design pattern**

- Singleton pattern is mostly used in multi-threaded and database applications. It is used in logging, caching, thread pools, configuration settings etc.



# Decorator Pattern

- A Decorator Pattern says that just "**attach a flexible additional responsibilities to an object dynamically**".
- Decorator pattern allows a user to add new functionality to an existing object without altering its structure. This type of design pattern comes under structural pattern as this pattern acts as a wrapper to existing class.
- This pattern creates a decorator class which wraps the original class and provides additional functionality keeping class methods signature intact.
- The Decorator Pattern is also known as **Wrapper**.

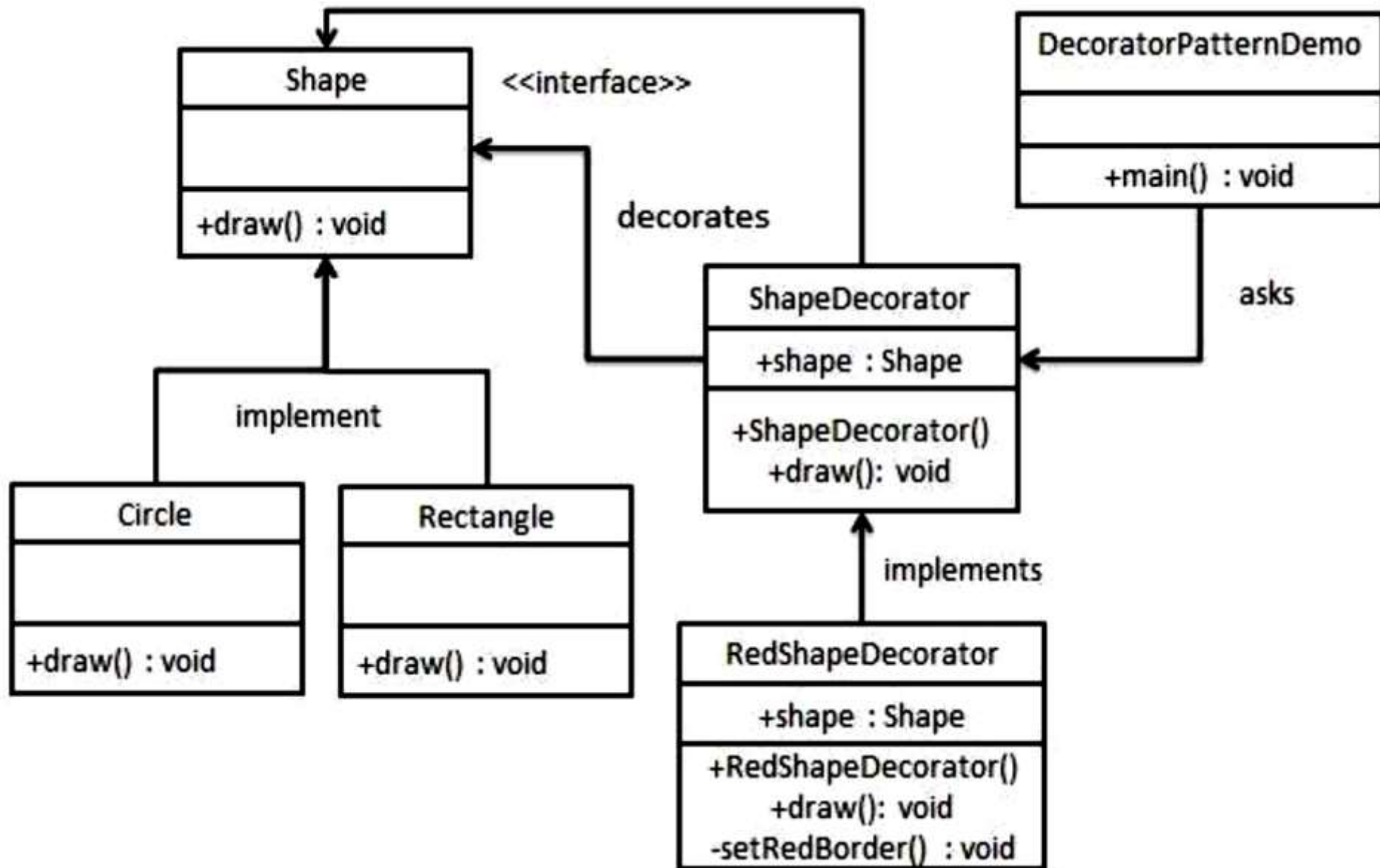
## **Advantage of Decorator Pattern**

- It provides greater flexibility than static inheritance.
- It enhances the extensibility of the object, because changes are made by coding new classes.
- It simplifies the coding by allowing you to develop a series of functionality from targeted classes instead of coding all of the behavior into the object.

## **Usage of Decorator Pattern**

It is used:

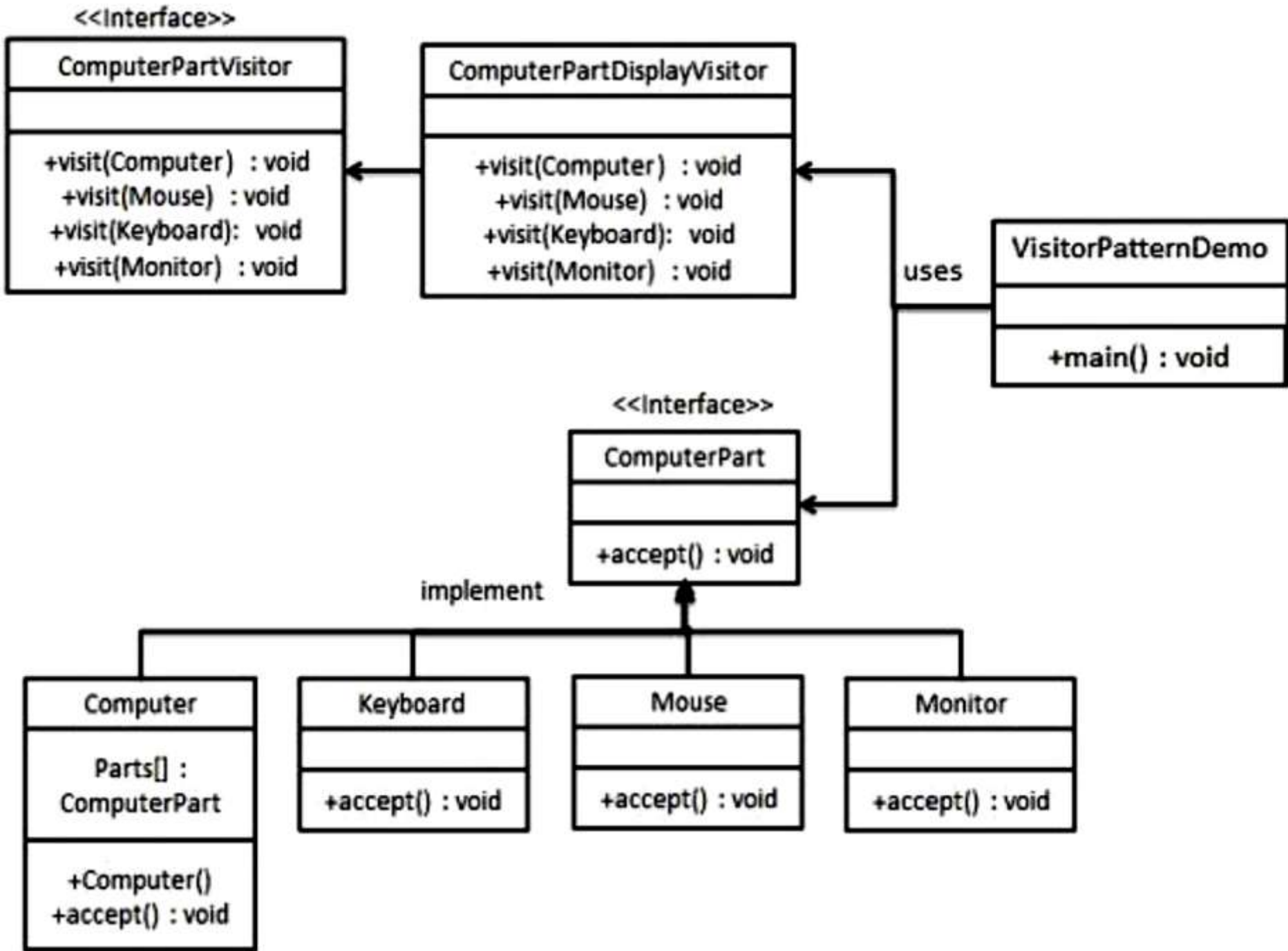
- When you want to transparently and dynamically add responsibilities to objects without affecting other objects.
- When you want to add responsibilities to an object that you may want to change in future.
- Extending functionality by sub-classing is no longer practical.



# Visitor Pattern

- In Visitor pattern, we use a visitor class which changes the executing algorithm of an element class. By this way, execution algorithm of element can vary as and when visitor varies. This pattern comes under behavior pattern category. As per the pattern, element object has to accept the visitor object so that visitor object handles the operation on the element object.
- Visitor pattern is used when we have to perform an operation on a group of similar kind of Objects. With the help of visitor pattern, we can move the operational logic from the objects to another class. For example, think of a Shopping cart where we can add different type of items (Elements). When we click on checkout button, it calculates the total amount to be paid. Now we can have the calculation logic in item classes or we can move out this logic to another class using visitor pattern.

- We are going to create a *ComputerPart* interface defining accept operation.  
*Keyboard, Mouse, Monitor* and *Computer* are concrete classes implementing *ComputerPart* interface. We will define another interface *ComputerPartVisitor* which will define a visitor class operations. *Computer* uses concrete visitor to do corresponding action.
- *VisitorPatternDemo*, our demo class, will use *Computer* and *ComputerPartVisitor* classes to demonstrate use of visitor pattern.



# Adapter Pattern

An Adapter Pattern says that just "**converts the interface of a class into another interface that a client wants**".

In other words, to provide the interface according to client requirement while using the services of a class with a different interface.

The Adapter Pattern is also known as **Wrapper**.

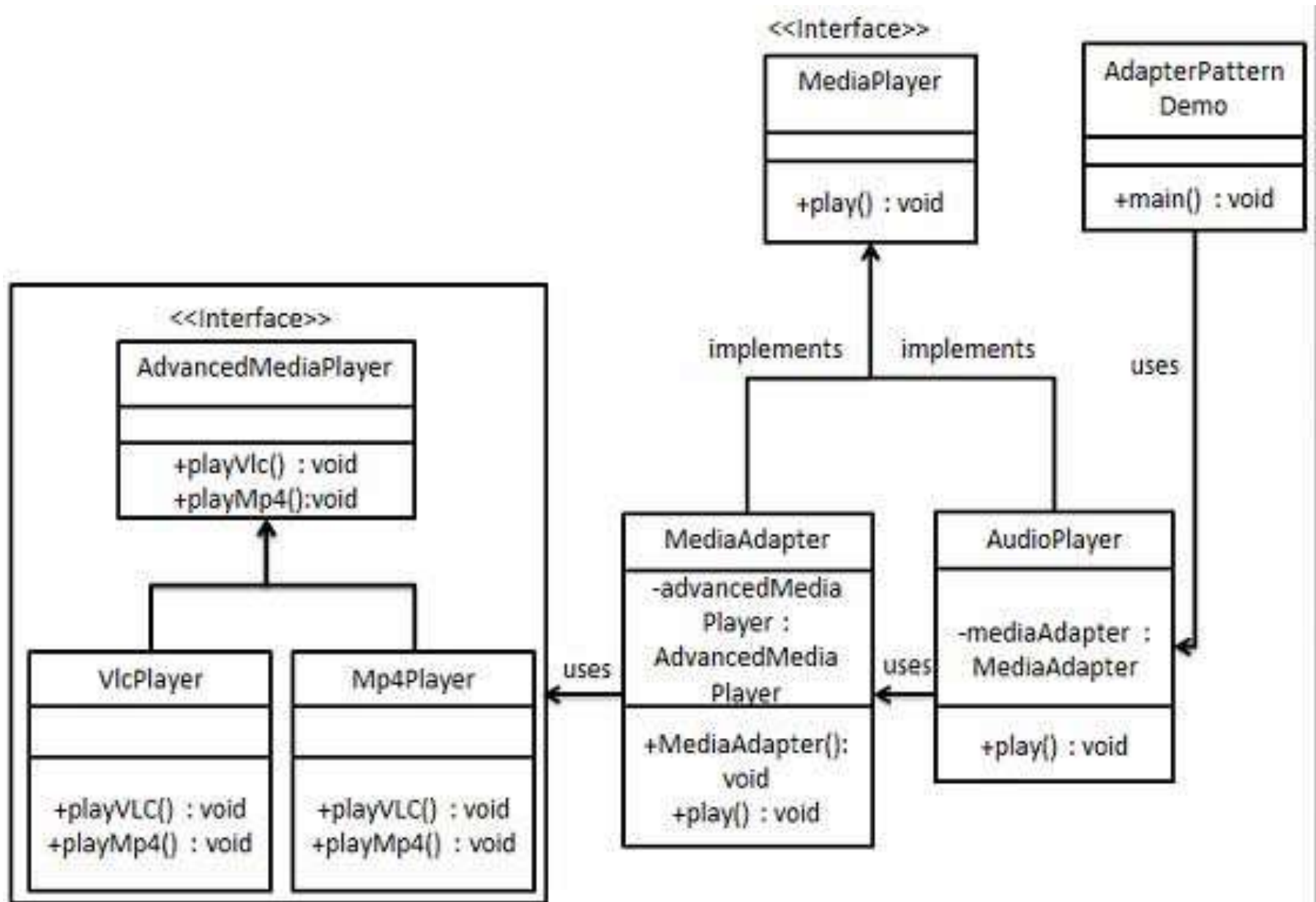
Advantage of Adapter Pattern

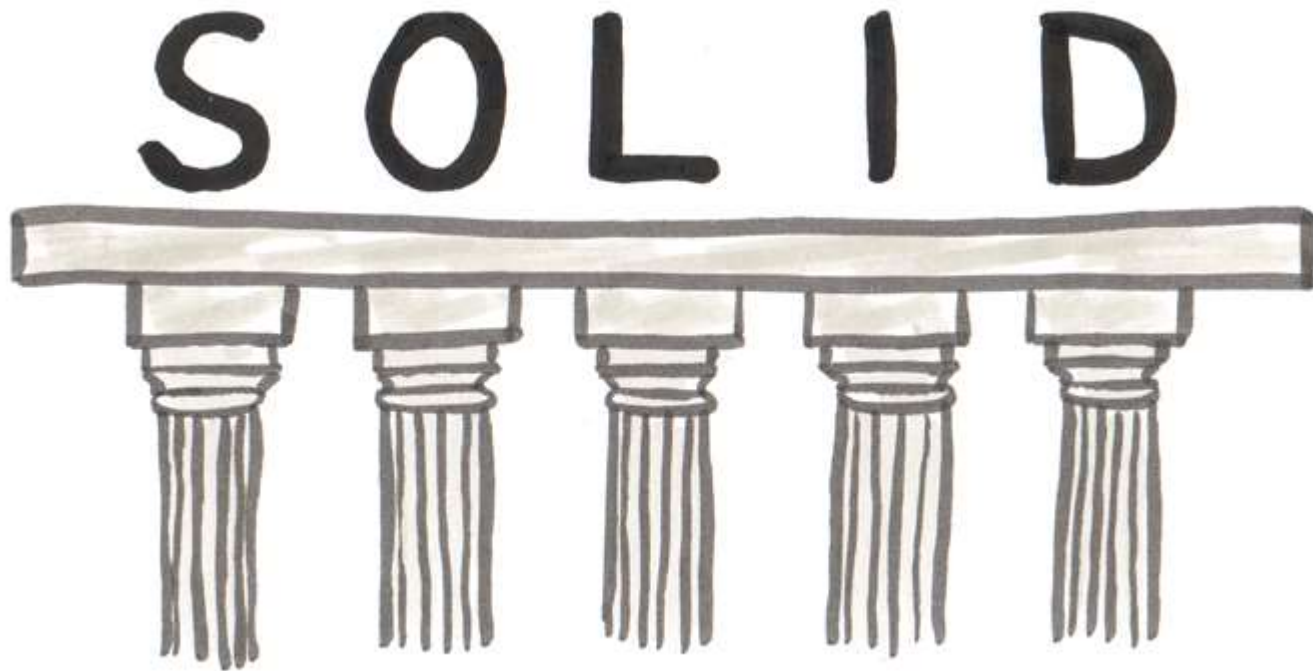
- It allows two or more previously incompatible objects to interact.
- It allows reusability of existing functionality.

It is used

- When an object needs to utilize an existing class with an incompatible interface.
- When you want to create a reusable class that cooperates with classes which don't have compatible interfaces.
- When you want to create a reusable class that cooperates with classes which don't have compatible interfaces.

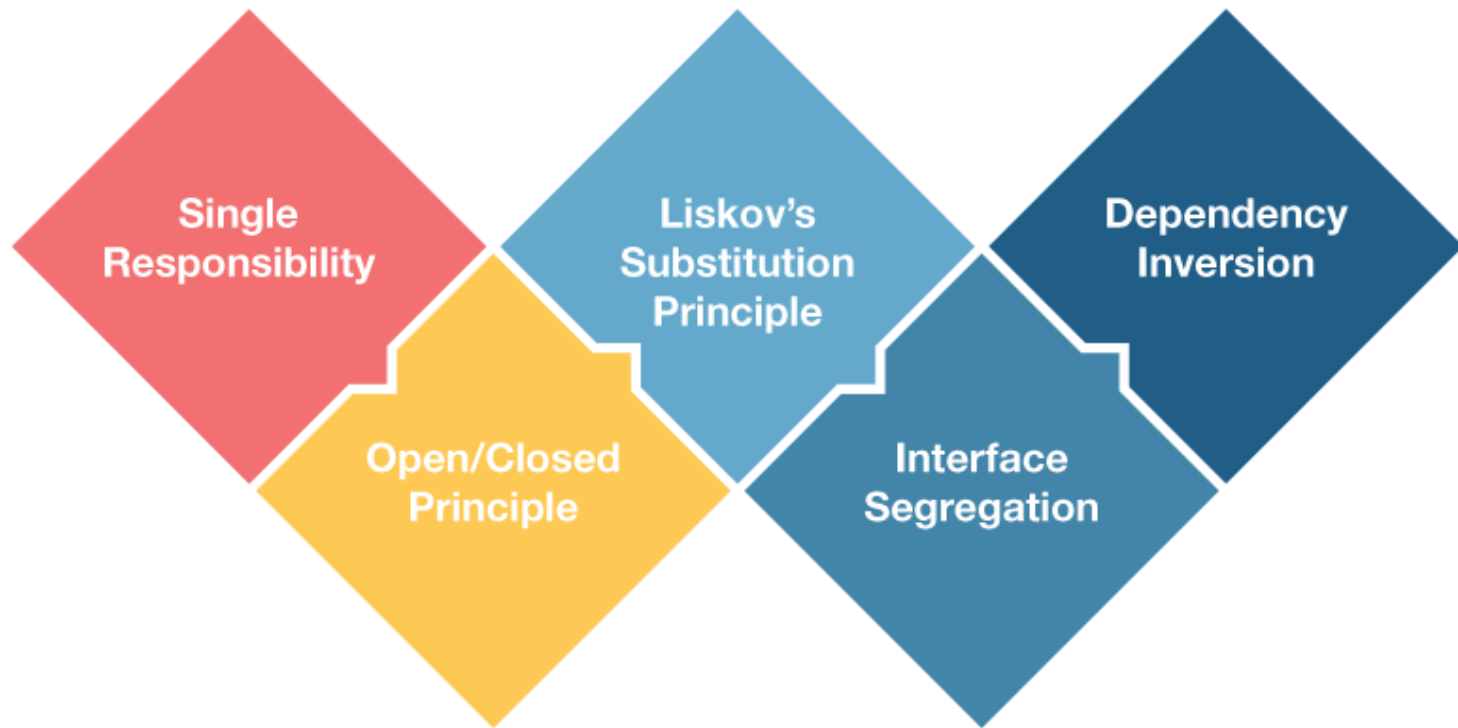
- **Target Interface:** This is the desired interface class which will be used by the clients.
- **Adapter class:** This class is a wrapper class which implements the desired target interface and modifies the specific request available from the Adaptee class.
- **Adaptee class:** This is the class which is used by the Adapter class to reuse the existing functionality and modify them for desired use.
- **Client:** This class will interact with the Adapter class.





- *SOLID* is an acronym for the first five object-oriented design (OOD).
- The SOLID principles were first conceptualized by Robert C. Martin in his 2000 paper, [\*Design Principles and Design Patterns\*](#).
- These principles establish practices that lend to developing software with considerations for maintaining and extending as the project grows. Adopting these practices can also contribute to avoiding code smells, refactoring code, and Agile or Adaptive software development.

# S.O.L.I.D.



# 1. Single Responsibility

This principle states that **a class should only have one responsibility. Furthermore, it should only have one reason to change.**

**How does this principle help us to build better software? Let's see a few of its benefits:**

**Testing** — A class with one responsibility will have far fewer test cases

**Lower coupling** — Less functionality in a single class will have fewer dependencies

**Organization** — Smaller, well-organized classes are easier to search than monolithic ones

## 2. Open for Extension, Closed for Modification

Simply put, **classes should be open for extension, but closed for modification.** In doing so, we stop ourselves from **modifying existing code** and causing potential new bugs in an otherwise happy application.

Of course, the **one exception to the rule is when fixing bugs in existing code.**

### 3. Liskov Substitution

Next up on our list is Liskov substitution, which is arguably the most complex of the 5 principles. Simply put, **if class *A* is a subtype of class *B*, then we should be able to replace *B* with *A* without disrupting the behavior of our program.**

## 4. Interface Segregation

The 'I' in SOLID stands for interface segregation, and it simply means that **larger interfaces should be split into smaller ones. By doing so, we can ensure that implementing classes only need to be concerned about the methods that are of interest to them.**

## 5. Dependency Inversion

The principle of Dependency Inversion refers to the decoupling of software modules. This way, instead of high-level modules depending on low-level modules, both will depend on abstractions.